

Hugging Face ↔ GitHub 同步完整操作手冊

本文件涵蓋:從 GitHub Codespaces 設定 SSH 金鑰、Clone Hugging Face repo、Push 到 GitHub 與 Hugging Face、以及用 GitHub Actions 做自動同步。最後附上常見錯誤與解法。

目錄

1. 前置準備:在 Codespaces 產生 SSH 金鑰
 2. 把 Public Key 貼到 GitHub
 3. 測試 SSH 連線
 4. 從 Hugging Face Clone Repo
 5. 建立 GitHub 空 Repo 並設定 Remote
 6. Push 到 GitHub
 7. 日常開發流程(雙邊 Push)
 8. 用 GitHub Actions 自動同步到 Hugging Face
 9. 常見錯誤與解法
-

1.前置準備:在 Codespaces 產生 SSH 金鑰

適合在公共場所、不想在本機留下金鑰痕跡時使用。

1.1 開啟 Codespace

- 到任一個 GitHub repo(或建一個新的空 repo)
- **Code → Codespaces → Create codespace on main**

1.2 產生金鑰

在 Codespace 的 Terminal 執行:

```
bash  
  
ssh-keygen -t ed25519 -C "your_email@example.com"
```

會依序詢問:

```
Enter file in which to save the key (/home/codespace/.ssh/id_ed25519):
```

→ 直接按 **Enter**(用預設路徑)

Enter passphrase (empty for no passphrase):

→ 直接按 **Enter**(留空,不設密碼)

Enter same passphrase again:

→ 再按一次 **Enter**

為什麼不設密碼? 這把金鑰通常是臨時、單一用途(搬 repo / 自動化同步),留空密碼在之後串接 GitHub Actions 時比較單純,不用處理密碼輸入的問題。Codespace 本身也是相對隔離的環境,風險較低。

執行完成後會看到:

```
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:xxxxxxxxxxxxxxxx your_email@example.com
```

`SHA256:xxxx...` 只是這把金鑰的指紋,方便肉眼核對用,**不需要複製到任何地方**。

2. 把 Public Key 貼到 GitHub

2.1 取得 Public Key 內容(不建議用 cat 手動選取,建議用 VS Code 開啟)

```
bash

code ~/.ssh/id_ed25519.pub
```

這會在 Codespaces 網頁版 VS Code 開一個分頁顯示內容。接著:

- 點進編輯畫面
- `Ctrl+A` 全選
- `Ctrl+C` 複製

複製完可以直接關閉這個分頁(點分頁右邊的 X,或 `Ctrl+W`),不會影響金鑰檔案本身。

內容看起來像這樣(一整行):

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
your_email@example.com
```

⚠ 絕對不要複製 `id_ed25519`(沒有 `.pub` 的檔案),那是私鑰,只有 `.pub` 結尾的檔案才能公開貼到網站上。

補充:`xclip` 這類指令在 Codespaces 這種沒有圖形介面的容器裡通常無法運作(需要連接系統剪貼簿),所以不建議在 Codespaces 使用,改用 VS Code 開啟複製即可。

2.2 貼到 GitHub

1. 前往 <https://github.com/settings/keys>
2. 點 **New SSH key**
3. Key type 選 **Authentication Key**(不是 Signing Key)
4. Title 隨意命名,例如 `codespaces-hf-sync`
5. 貼上同一段內容 → 儲存

3. 測試 SSH 連線

```
bash
ssh -T git@hf.co
ssh -T git@github.com
```

第一次連線會出現類似:

```
The authenticity of host 'github.com (20.205.243.166)' can't be established.
...
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

直接輸入 `yes` 並按 Enter(這是連到官方伺服器的正常提示,可放心信任)。

成功的話會看到:

```
Hi your_username! You've successfully authenticated, but GitHub does not
provide shell access.
```

兩邊(HF 與 GitHub)都要測試成功才能繼續下一步。

4. 從 Hugging Face Clone Repo

4.1 安裝 Git LFS(如果 repo 裡有模型權重等大檔案)

```
bash
git lfs install
```

4.2 Clone(依 repo 類型選一個)

```
bash

# Model
git clone git@hf.co:username/repo-name

# Dataset
git clone git@hf.co:datasets/username/repo-name

# Space
git clone git@hf.co:spaces/username/repo-name
```

例如:

```
bash

git clone git@hf.co:spaces/monzid1/E_commerce
```

Clone 完成後,進入資料夾:

```
bash

cd E_commerce
```

5. 建立 GitHub 空 Repo 並設定 Remote

5.1 在 GitHub 網頁建立新 repo

- **不要**勾選自動產生 README
- **不要**加 .gitignore
- **不要**加 License

保持完全空白,避免等一下 push 時產生無關的 commit 歷史衝突。

▮ 如果想加 `.gitignore`,可以等搬移完成後再手動補上,不影響已經 commit 的檔案。

5.2 加上 GitHub 當作新的 remote

```
bash

git remote add github git@github.com:你的帳號/repo-name.git
```

確認 remote 設定:

```
bash
```

```
git remote -v
```

應該會看到:

```
github  git@github.com:你的帳號/repo-name.git (fetch)
github  git@github.com:你的帳號/repo-name.git (push)
origin  git@hf.co:spaces/username/repo-name (fetch)
origin  git@hf.co:spaces/username/repo-name (push)
```

`origin` 指向 Hugging Face(clone 時自動設定),`github` 指向 GitHub(手動加的)。

6. Push 到 GitHub

```
bash

git branch -M main
git push github main
```

如果有 LFS 大檔案,額外推送:

```
bash

git lfs push github --all
```

如果有其他分支或 tag:

```
bash

git push github --all
git push github --tags
```

 GitHub 免費帳號的 LFS 額度通常只有 **1GB 儲存 + 1GB/月頻寬**,如果模型檔案很大,可能會超過限制。建議大型模型檔案還是留在 Hugging Face,GitHub 只放程式碼。

7. 日常開發流程(雙邊 Push)

在這個 Codespace(或未來重新開啟後)編輯程式碼:

```
bash
```

```
# 1. 修改檔案後,加入並 commit
git add .
git commit -m "說明這次改了什麼"

# 2. 推到 GitHub(平常開發、版控用)
git push github main

# 3. 推到 Hugging Face(讓 Space 重新部署更新)
git push origin main
```

一次推兩邊:

```
bash

git push origin main && git push github main
```

注意:這兩個 push 是各自獨立的動作,不會自動同步。想兩邊都更新,就要各自執行一次 push,除非設定第 8 節的自動化。

關於「從 GitHub repo 開 Codespace」的差異

	手動 clone HF 再加 github remote(本文做法)	直接從 GitHub repo 頁面開 Codespace
<code>origin</code> 指向	Hugging Face	GitHub
是否要手動加 remote	是	否(自動設定好)
Push 到 GitHub 指令	<code>git push github main</code>	<code>git push</code> (預設就是 GitHub)

若之後想要更直覺的開發習慣,可以改成直接從 <https://github.com/你的帳號/repo-name> 頁面點 **Code** → **Codespaces** → **Create codespace**,這樣「codespace 裡的檔案」就直接對應 GitHub repo,再額外手動加一個指向 HF 的 remote(例如取名 `hf`)即可。

8. 用 GitHub Actions 自動同步到 Hugging Face

讓你只要 `git push github main`,就自動觸發同步到 Hugging Face,不用手動 push 兩次。

8.1 把 HF 的私鑰存到 GitHub Secrets

取得私鑰內容:

```
bash
```

```
cat ~/.ssh/id_ed25519
```

複製整段(含 -----BEGIN OPENSSH PRIVATE KEY----- 到 -----END OPENSSH PRIVATE KEY-----)。

到 GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

- Name: `HF_SSH_PRIVATE_KEY`
- Value: 貼上私鑰完整內容

⚠ 私鑰是敏感資訊,只能放在 GitHub Secrets 這種加密儲存的地方,不要貼在一般程式碼、Issue 或 README 裡。

8.2 建立 Workflow 檔案

在 repo 裡建立 `.github/workflows/sync-to-hf.yml`:

yaml

```
name: Sync to Hugging Face

on:
  push:
    branches:
      - main
  workflow_dispatch:    # 也允許手動觸發

jobs:
  sync:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout GitHub repo
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          lfs: true

      - name: Setup SSH for Hugging Face
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.HF_SSH_PRIVATE_KEY }}" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan hf.co >> ~/.ssh/known_hosts

      - name: Push to Hugging Face
        run: |
          git remote add hf git@hf.co:spaces/monzid1/E_commerce
          git push hf HEAD:main --force
```

把 `git@hf.co:spaces/monzid1/E_commerce` 換成你實際的 HF repo(Model 用 `git@hf.co:username/repo`, Dataset 用 `git@hf.co:datasets/username/repo`)。

8.3 之後的使用方式

只要:

```
bash

git push github main
```

GitHub Actions 就會自動:

1. Checkout 你剛 push 的內容
2. 用 Secrets 裡的私鑰連到 Hugging Face
3. 自動 push 過去,觸發 Space 重新部署

可以到 GitHub repo 的 **Actions** 分頁查看執行紀錄與 log。

9. 常見錯誤與解法

9.1 `ssh: connect to host github.com port 22: Connection refused`

公司或學校網路可能封鎖 22 port。解法是改用 443 port 連線,在 `~/.ssh/config` 加入:

```
Host github.com
  Hostname ssh.github.com
  Port 443
  User git
```

9.2 `Permission denied (publickey)`

- 確認 public key 真的有貼到對的帳號設定頁面(HF / GitHub)
- 確認本機用的是同一把私鑰:

```
bash

ssh -i ~/.ssh/id_ed25519 -T git@github.com
```

- 確認檔案權限正確:

```
bash

chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub
```

9.3 `! [rejected] main -> main (fetch first)`

GitHub 端已有不相關的 commit 歷史(例如建 repo 時自動產生了 README)。

方法一:直接覆蓋(建議,若 GitHub 端內容不重要)

```
bash

git push github main --force
```

方法二:合併保留兩邊內容

```
bash
```

```
git pull github main --allow-unrelated-histories
# 若有衝突,手動編輯解決後:
git add .
git commit -m "Merge GitHub initial commit with Hugging Face content"
git push github main
```

9.4 LFS 檔案推送失敗 / 超過額度

GitHub 免費帳號 LFS 額度為 1GB 儲存 + 1GB/月頻寬。若模型檔案過大:

- 考慮只把程式碼放 GitHub,大型權重檔留在 Hugging Face
- 或升級 GitHub LFS 方案(Data Pack)
- 或在 `.gitattributes` / `.gitignore` 中排除大檔案後再推送

9.5 Secret 貼入 GitHub/HF 時提示「有其他符號無法存檔」

通常是複製時夾帶了不可見的控制字元(常發生於在終端機用滑鼠拖曳選取)。

解法:改用 VS Code 開啟檔案再複製,較乾淨:

```
bash

code ~/.ssh/id_ed25519
```

`(Ctrl+A)` 全選 → `(Ctrl+C)` 複製,再貼到目標欄位。

9.6 誤刪 Terminal 分頁,擔心金鑰不見

- 若只是關掉 Terminal 分頁:金鑰檔案完全不受影響,重新開一個新 Terminal(`Terminal` → `New Terminal`),再執行 `(ls -al ~/.ssh)` 確認金鑰還在即可。
- 若是整個 Codespace 被刪除:因 Codespace 是暫時性容器,未備份的金鑰會遺失,需要重新執行 `(ssh-keygen)` 並重新在 HF / GitHub 貼新的 public key。

9.7 GitHub Actions 執行失敗: `Host key verification failed`

Workflow 裡忘記加入 `(ssh-keyscan)` 註冊 known_hosts,確認 workflow 內有這一行:

```
yaml

ssh-keyscan hf.co >> ~/.ssh/known_hosts
```

附錄:指令速查表

目的	指令
產生 SSH 金鑰	<code>ssh-keygen -t ed25519 -C "email"</code>
查看 public key	<code>code ~/.ssh/id_ed25519.pub</code>
查看 private key	<code>code ~/.ssh/id_ed25519</code>
測試 HF 連線	<code>ssh -T git@hf.co</code>
測試 GitHub 連線	<code>ssh -T git@github.com</code>
Clone HF Space	<code>git clone git@hf.co:spaces/user/repo</code>
加 GitHub remote	<code>git remote add github git@github.com:user/repo.git</code>
查看 remote	<code>git remote -v</code>
Push 到 GitHub	<code>git push github main</code>
Push 到 HF	<code>git push origin main</code>
強制覆蓋 push	<code>git push github main --force</code>
LFS 檔案清單	<code>git lfs ls-files</code>
LFS 推送	<code>git lfs push github --all</code>